Migration of Applications

From Spring Boot to Quarkus

Aurea Munoz Hernandez

Charles Moulliard

Feb-24-2026

Agenda

- OpenRewrite: Migration Swiss knife toolbox
- New project: Quarkus Rewrite Client
- What about Migrating Applications
- The Quarkus Migration Tool project
- When we reach the limit: AI
- Wrap-up & Q/A

OpenRewrite

Migration Swiss knife toolbox

What is OpenRewrite

- OpenRewrite documentation
- Huge collection of **recipes** = Java classes executed through visitor pattern
- Can be used to **transform** OR **analyze**
- Supports `"dryRun"` or `"run"` mode
- Parsers available when project scanned: java, properties, YAML, json, etc
- Executed using **Maven/Gradle plugin**

Example

```
mvn org.openrewrite.maven:rewrite-maven-plugin:dryRun \
  -Drewrite.activeRecipes=org.openrewrite.java.format.AutoFormat
```

... Limitations

- Process could become really slow when several `mvn` commands must be executed
- No easy way to design integration tests
- No API or Library
- Fetching & analysing CSV files generated could be tedious

Quarkus Rewrite Client

https://github.com/snowdrop/rewrite-client

Quarkus Picocli Client & Java Library designed around OpenRewrite

Bypasses most of the limitations described

Demo 0

To install the client, simply ... use jbang !

```
# Latest version
jbang app install rewrite@snowdrop/rewrite-client

# Specific version
jbang app install rewrite@snowdrop/rewrite-client/0.2.11

# When coding use your local catalog pointing to the uber jar
jbang app install --catalog jbang-catalog-dev.json rewrite
```

Demo 1

Play with the Rewrite Client

```
pushd test-project/simple

# Using mvn openrewrite
mvn org.openrewrite.maven:rewrite-maven-plugin:dryRun \
  -Drewrite.activeRecipes=org.openrewrite.java.format.AutoFormat

# Simple recipe in dryn or run mode
rewrite . -r org.openrewrite.java.format.AutoFormat

# With Java recipe's fields to be set
rewrite . \
  -r org.openrewrite.java.search.FindAnnotations \
  -o annotationPattern=@org.springframework.boot.autoconfigure.SpringBootApplic

# With additional recipes jar files or GAVs
rewrite /path/to/project --jar /path/to/recipes.jar -r com.example.MyRecipe
popd
```

Demo 2

Use recipes declared in a YAML

```
# AutoFormat, NormalizeTabsOrSpaces & AddLicense
rewrite . -c rewrite.yml
```

Using the Java Library

```
import dev.snowdrop.openrewrite.RewriteService;
import dev.snowdrop.openrewrite.Config;
import dev.snowdrop.openrewrite.ResultsContainer;

Config cfg = new Config();
cfg.setAppPath(Paths.get("/path/to/your/project"));
cfg.setRecipes(List.of("org.openrewrite.java.search.FindAnnotations"));
cfg.setRecipeOptions(Set.of("annotationPattern=org.springframework.boot.autocor

RewriteService scanner = new RewriteService(cfg);
scanner.init();
ResultsContainer results = scanner.run();

// Access results
RecipeRun run = results.getRecipeRuns().get("org.openrewrite.java.search.FindAr
// Process datatables, changesets, etc.
```

Client Test case

```
@QuarkusTest
public class FindAnnotationTest extends BaseTest {

    @Test
    void shouldFindAnnotation() throws Exception {
        List rows = findDataTableRows(run, "SearchResults", SearchResults.Row.c
        assertEquals(1, rows.size());

        SearchResults.Row record = rows.getFirst();
        assertEquals("src/main/java/com/todo/app/AppApplication.java", record.g
        assertEquals("@SpringBootApplication", record.getResult());
        assertEquals("Find annotations `org.springframework.boot.autoconfigure.
```

What about Migrating Applications ?

- OpenRewrite tool ⇒ perfect to execute YAML recipes
- Scenarios can be **really complex** even when migrating some modules, REST annotations, etc
- Examples
  - Quarkus RESTeasy migration
  - Some modules of SpringBoot to Quarkus
  - Moving the Spring Boot starters to Quarkus

Pain Points

- Execute and hope it works
- No conditional logic
- No eligibility validation
- **OpenRewrite is extremely powerful, but it's designed to execute transformations, not to orchestrate complex migrations.**
- **We need an intelligence layer !**

The Quarkus Migration Tool Project - mtool

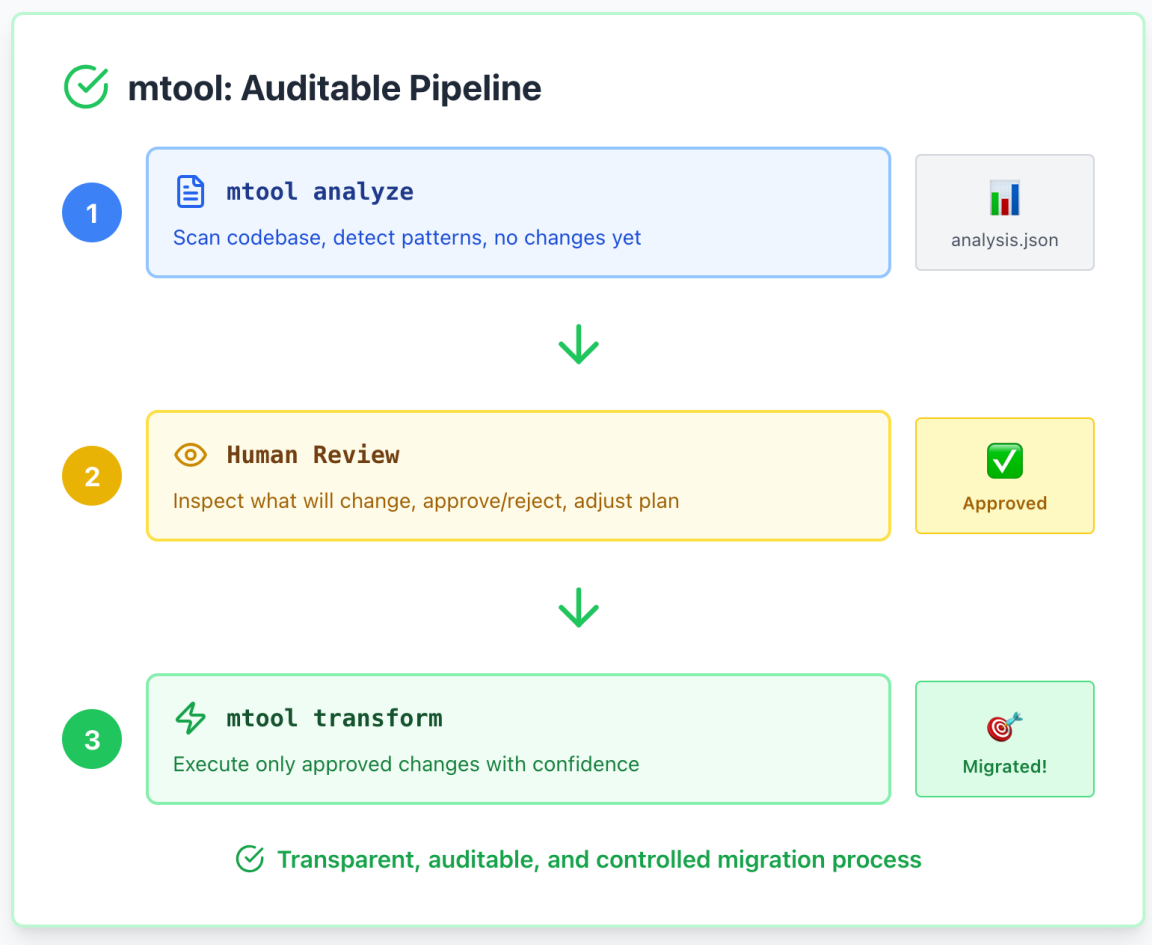
https://github.com/snowdrop/migration-tool

Quarkus Client able to **analyze**, **generate a migration plan**, and **transform**

This is still a PoC running the rewrite client but with new features

Inspired by Konveyor kantra, IBM Java Binary Scanner tool, etc

Analyze → Review → Transform pipeline



Use rules and instructions

```yaml
- message: "SpringBoot to Quarkus."
  ruleID: 010-springboot-replace-bom-quarkus
  when:
    condition: java.annotation is 'org.springframework.boot.autoconfigure.SpringBootApplication'
  order: 1
  instructions:
    ai: <1 item>
    manual: <1 item>
    openrewrite:
      - name: Replace the SpringBoot parent dependency with Quarkus BOM within the pom.xml file
        description: Replace the SpringBoot parent dependency with Quarkus BOM within the pom.xml file.
        recipeList:
          - org.openrewrite.maven.AddManagedDependency:
              groupId: io.quarkus.platform
              artifactId: quarkus-bom
              version: 3.31.3
              type: pom
              scope: import
              addToRootPom: false
          - org.openrewrite.maven.AddDependency:
              groupId: io.quarkus
              artifactId: quarkus-core
              version: 3.31.3
```

What to search about !

Instructions to be executed by the tool

Demo 3

Spring Boot Todo → Quarkus Todo

```
# Scan and analyse using (pre)conditions
mtool analyze ./applications/spring-boot-todo-app \
      -r ./cookbook/rules/quarkus-spring-precondition
mtool analyze ./applications/spring-boot-todo-app \
      -r ./cookbook/rules/quarkus-spring

# Visualize the matching's ascii table generated in the console
# Quick review of the report = migration plan enerated with instructions
```

Demo 3

```
# Show that the Spring Boot Todo app is working

mvn clean package spring-boot:run
firefox localhost:8081
```

Demo 3

```
# Dry run - generates patch files
mtool transform ./applications/spring-boot-todo-app \
  -p openrewrite --dry-run

# Apply transformation
mtool transform ./applications/spring-boot-todo-app \
  -p openrewrite
```

Demo 3

Launch the migrated application

```
mvn clean package -DskipTests quarkus:dev
firefox http://localhost:8080/q/dev-ui/extensions
```

```
# Test the API
curl http://127.0.0.1:8080/home

curl -X POST http://localhost:8080/home \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Prepare migration to Quarkus. Task 1",
    "description": "Migrating Spring Boot TO DO to Quarkus. Task 1",
    "dueDate": "2025-12-24"
  }'

curl -X POST http://localhost:8080/home \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Prepare migration to Quarkus. Task 2",
    "description": "Migrating Spring Boot TO DO to Quarkus. Task 2",
    "dueDate": "2025-12-24"
  }'
```

Demo 4

A Quarkus to Quarkus Migration

```
# Scan and analyse using (pre)conditions
mtool analyze ./applications/quarkus-resteasy-classic-app -r ./cookbook/rules/c

# Visualize the matching's ascii table generated in the console
# Quick review of the report = migration plan with instructions
```

Demo 4

```
# Dry run - generates patch files
mtool transform ./applications/quarkus-resteasy-classic-app \
  -p openrewrite --dry-run

# Apply transformation
mtool transform ./applications/quarkus-resteasy-classic-app \
  -p openrewrite
```

Demo 4

Launch the migrated application

```
mvn clean quarkus:dev
firefox http://localhost:8080/q/dev-ui/extensions
```

```
# Test the API
curl http://localhost:8080/books

curl -X POST http://localhost:8080/books \
  -H "Content-Type: application/json" \
  -d '{
  "title": "Quarkus in Action",
  "author": "Martin Štefanko and Jan Martiška",
  "dueDate": "2022-03-10",
  "isbn": "9781633438958",
  "publicationYear": 2025
}'
```

Why not just OpenRewrite?

OpenRewrite is excellent for code transformation but enterprise migrations need more.

- mtool adds more flexibility.
- OpenRewrite powers the automated transformations.

We don't replace OpenRewrite. We complete it.

What Else ?

Mtool is still a POC but could be improved a lot to:

- Generate the rules using a designer (aka Apache camel routes & https://kaoto.io/)
- Integrate a simple **orchestration engine** to better handle the instructions
- Be able to support to switch between modes instructions: openrewrite and/or AI

Demo 5

AI-Assisted Migration

Wrap-up & Q/A

Thank you!